Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Дисциплина
«Технологии виртуализации»

Лабораторная работа №3

Знакомство с Docker

Выполнил: Малышев Никита Александрович
Группа: P3412
Проверил: Адмакин Максим Александрович

Санкт-Петербург
2026 год

Содержание

1. Цель 3
2. Установка Docker и VirtualBox 3
3. Запуск тестового контейнера 3
4. Контейнер с Nginx и пользовательской страницей 3
5. Контейнер с MariaDB 5
6. Контейнер с NextCloud 6
7. Контейнер с phpvirtualbox 8
8. Настройка окружения с помощью Docker Compose 10
9. Регистрация на Docker Hub 13
10. Публикация образа на Docker Hub 13
11. Запуск приложения из Docker Hub 14
12. Основные команды Docker 15
13. Работа с Portainer 16
    - 13.1. Запуск 16
    - 13.2. Возможности Portainer 17
        - 13.2.1. Мониторинг 17
        - 13.2.2. Администрирование 18
        - 13.2.3. Преимущества 18
14. Вывод 18

1. Цель

Изучить основы работы с Docker, основные команды, создание и запуск контейнеров, запуск и управление нескольких контейнеров, регистрацию на DockerHub.

2. Установка Docker и VirtualBox

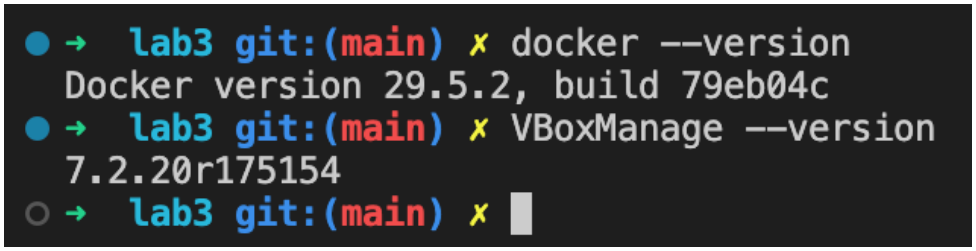


Рис. 1. Docker и VirtualBox установлены

3. Запуск тестового контейнера

Попробуем запустить тестовый контейнер `hello-malyshev`.

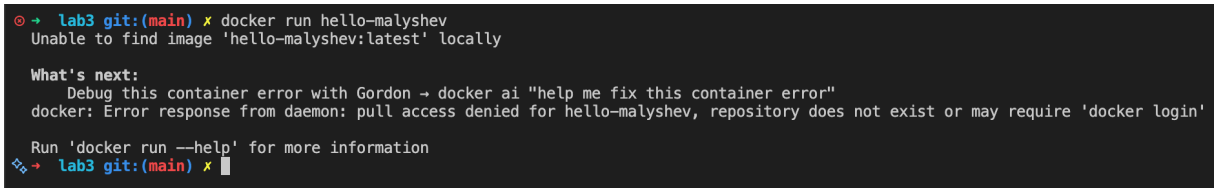


Рис. 2. Невозможно найти образ

Мы получаем ошибку, так как образа hello-malyshev не существует как локально, так и в удаленном репозитории образов. Исправим это, создав образ для hello-malyshev.

4. Контейнер с Nginx и пользовательской страницей

Создан каталог home/www/html, в нём размещён HTML-файл `index.html`. Написан `Dockerfile` на базе образа `nginx:latest`, который пробрасывает порт 8080 и копирует страницу.

```
➜  lab3 git:(main) ✗ cat home/www/html/index.html
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Информация</title>
</head>
<body>
    <h1>Студент: Н. А. Малышев</h1>
    <p>Контейнер успешно запущен.</p>
</body>
</html>%
➜  lab3 git:(main) ✗ cat home/www/html/Dockerfile
FROM nginx:alpine
WORKDIR /usr/share/nginx/html
COPY index.html .
EXPOSE 8080
CMD ["nginx", "-g", "daemon off;"]
➜  lab3 git:(main) ✗
```

Рис. 3. Содержимое Dockerfile и HTML-страницы

Образ собран и запущен с монтированием тома.

```
docker build -t my-nginx home/www/html
docker run -d -p 8080:80 -v /Users/admin/ws/docs/lab3/home/www/html:/usr/share/
nginx/html:ro --name my-webserver my-nginx
open http://localhost:8080
```

```
➜  lab3 git:(main) ✗ docker build -t my-nginx home/www/html
[+] Building 0.7s (8/8) FINISHED                                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                                0.0s
 => => transferring dockerfile: 150B                                                                                                0.0s
 => [internal] load metadata for docker.io/library/nginx:alpine                                                                     0.6s
 => [internal] load .dockerignore                                                                                                   0.0s
 => => transferring context: 2B                                                                                                     0.0s
 => [1/3] FROM docker.io/library/nginx:alpine@sha256:b2f80f8f88146b0772fdcc7d92cea621aa8c2889fa8d07ed61d4884fcea53e09               0.0s
 => => resolve docker.io/library/nginx:alpine@sha256:b2f80f8f88146b0772fdcc7d92cea621aa8c2889fa8d07ed61d4884fcea53e09               0.0s
 => [internal] load build context                                                                                                   0.0s
 => => transferring context: 32B                                                                                                    0.0s
 => CACHED [2/3] WORKDIR /usr/share/nginx/html                                                                                      0.0s
 => CACHED [3/3] COPY index.html .                                                                                                  0.0s
 => exporting to image                                                                                                              0.0s
 => => exporting layers                                                                                                             0.0s
 => => exporting manifest sha256:ae20425bc0fec758d8c3591d4d13ad0d3611e1c5ac3e27894d78bcf2df314890                                   0.0s
 => => exporting config sha256:0a1851ab7fb3d60430c7f2621df6ec1b2224fe2b3de401f95d9267cbbeda2f01                                     0.0s
 => => exporting attestation manifest sha256:53980a30aa6ee965c3511aafd1eb3a245106017b77ef1cf9bc6fcebc3cea6e50                       0.0s
 => => exporting manifest list sha256:5c3e4f244248bda523cb04a6cc5d28725292675262bfcc782589a593aafc82ce                              0.0s
 => => naming to docker.io/library/my-nginx:latest                                                                                  0.0s
 => => unpacking to docker.io/library/my-nginx:latest                                                                               0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/ilzsd2p9f92a9194vdt7bxt6v
➜  lab3 git:(main) ✗ docker run -d -p 8080:80 -v /Users/admin/ws/docs/lab3/home/www/html:/usr/share/nginx/html:ro --name my-webserver my-nginx
d9963dcca1530965cd93f42a25e3d1b9013ef898a22885f722fb4abd800b7874
➜  lab3 git:(main) ✗
```

Рис. 4. Сборка и запуск образа

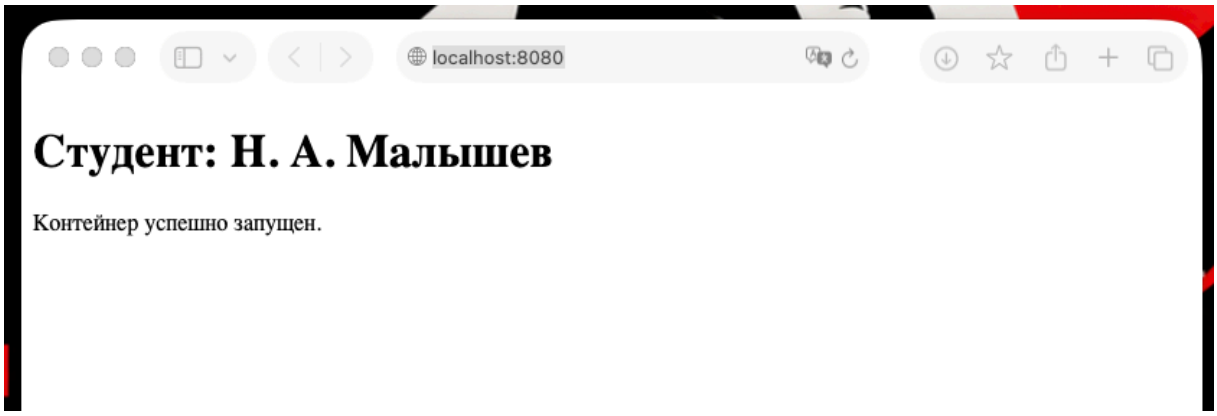


Рис. 5. Результат работы веб-сервера в браузере

5. Контейнер с MariaDB

Запущен контейнер `mariadb` с монтированием тома `home/DB` и пробросом порта 3306.

```
docker run -d --name my-mariadb \
  -v /Users/admin/ws/docs/lab3/home/db:/var/lib/mysql \
  -e MARIADB_ROOT_PASSWORD=admin \
  -p 3306:3306 \
  mariadb:12.1.1-rc
```

```
● →  lab3 git:(main) ✗ docker run -d --name my-mariadb \
    -v /Users/admin/ws/docs/lab3/home/db:/var/lib/mysql \
    -e MARIADB_ROOT_PASSWORD=admin \
    -p 3306:3306 \
    mariadb:12.1.1-rc
  Unable to find image 'mariadb:12.1.1-rc' locally
  12.1.1-rc: Pulling from library/mariadb
  ced495fd5ef0: Pull complete
  0b2d497af9d3: Pull complete
  05ed69fda2c2: Pull complete
  4692c387f0a1: Pull complete
  3e0873dbd2d3: Pull complete
  d4e9dfc044a9: Pull complete
  980beedee4ad: Pull complete
  97dd3f0ce510: Pull complete
  3e84f74ff2d8: Download complete
  43c78877a203: Download complete
  Digest: sha256:f18fab8692186f1a553ffd859728734ec91119d21cfa39cdf1b2f59265c972de
  Status: Downloaded newer image for mariadb:12.1.1-rc
  960a98abfde11ac475a14eb6d92076617c70ded3640953966a1e4c8e0a3f8ac1
○ →  lab3 git:(main) ✗
```

Рис. 6. Запуск mariadb

Через `docker exec` выполнено подключение к СУБД и создана база данных с именем `malyshev`

```
docker exec -it my-mariadb mariadb -u root -padmin -e "CREATE DATABASE malyshev;
SHOW DATABASES;"
```



Рис. 7. Создание базы данных в MariaDB через терминал контейнера

В итоге база данных сохранилась как отдельная папка внутри хостовой директории `home/DB`, которая смонтирована внутрь контейнера.

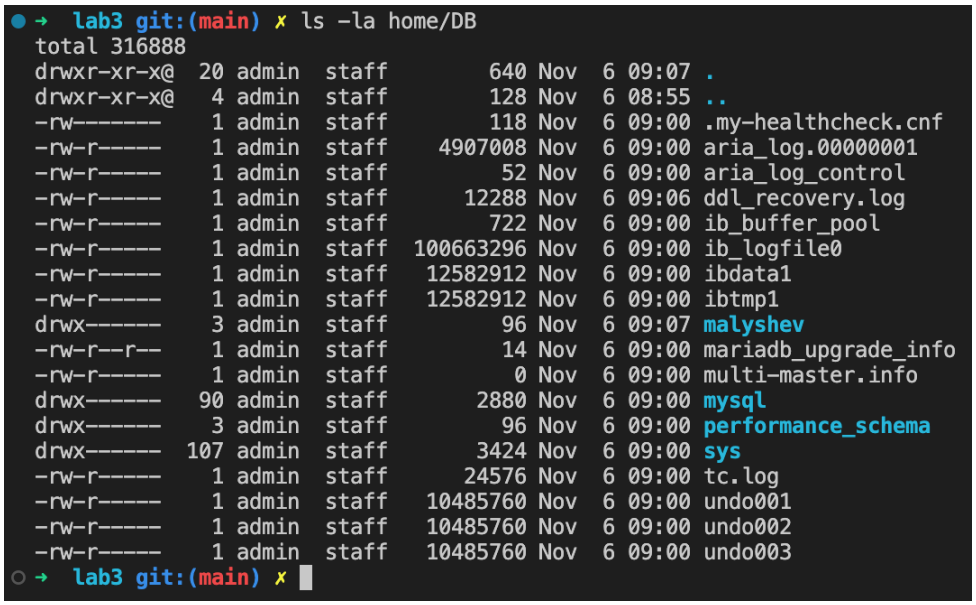


Рис. 8. Содержимое home/DB

6. Контейнер с NextCloud

Запущен официальный образ `nextcloud` с пробросом порта 8088 и монтированием каталога `home/www/NextCloud`.

```
docker run -d --name my-nextcloud \
  -v /Users/admin/ws/docs/lab3/home/www/NextCloud:/var/www/html \
  -p 8088:80 \
  nextcloud:latest
```

```
➜  lab3 git:(main) ✗ docker run -d --name my-nextcloud \
  -v /Users/admin/ws/docs/lab3/home/www/NextCloud:/var/www/html \
  -p 8088:80 \
  nextcloud:latest
732d87caf38098c493ad2b94edf3fdd3ccfe35815493b52b9ff3aeb4f7c87400
➜  lab3 git:(main) ✗
```

Рис. 9. Создание контейнера с NextCloud

Логин и пароль от NextCloud – admin:admin.

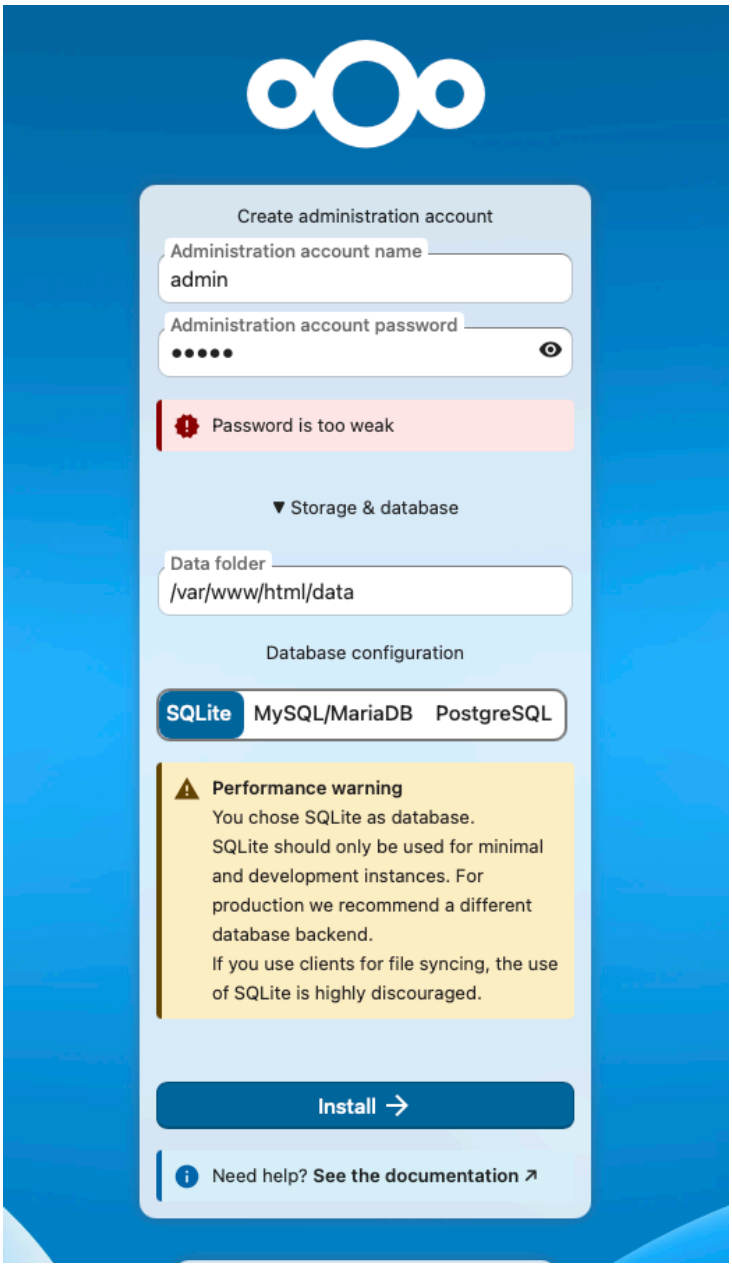


Рис. 10. Настройка NextCloud

Через веб-интерфейс зарегистрированы два пользователя: студент Н. А. Малышев и преподаватель М. А. Адмакин.

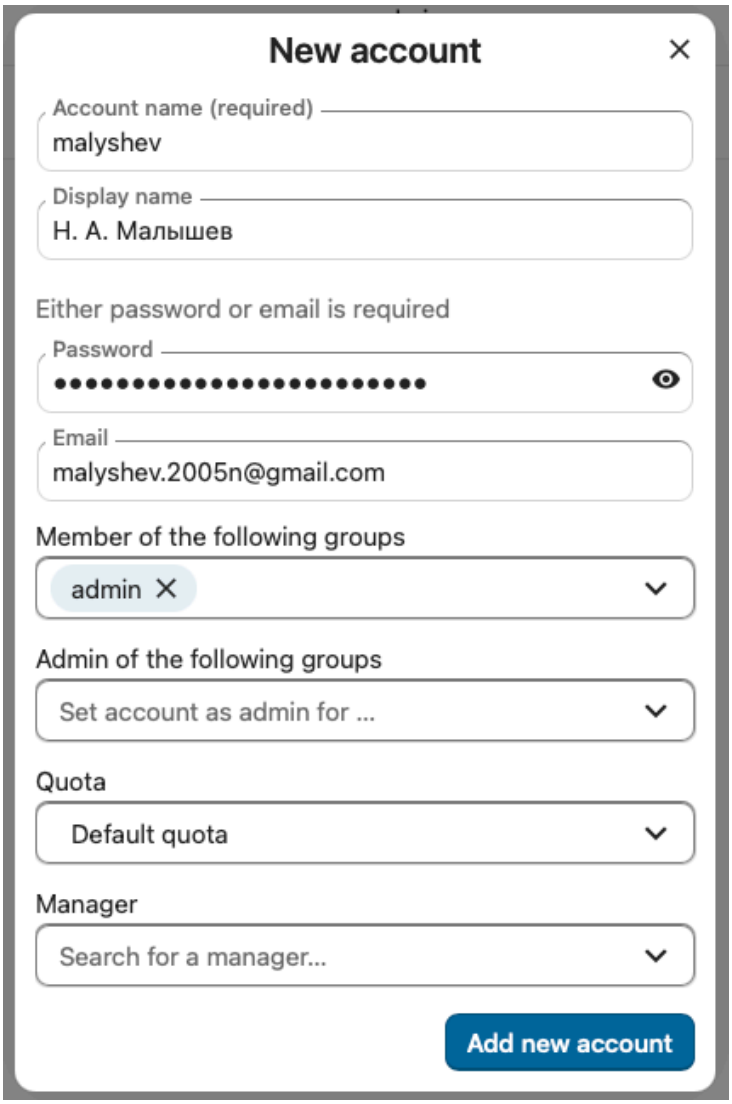


Рис. 11. Создание пользователя

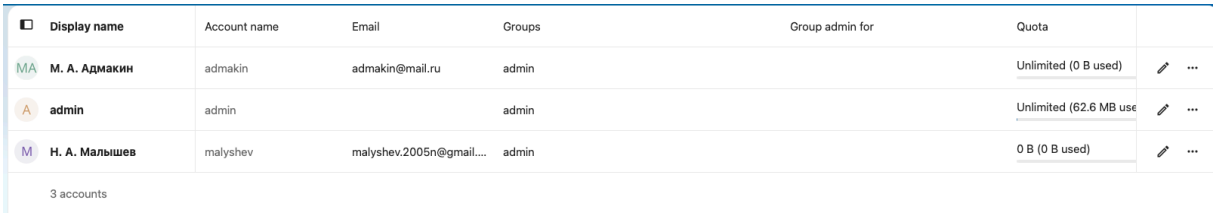


Рис. 12. Раздел Accounts в NextCloud

7. Контейнер с phpvirtualbox

Сначала запустим `vboxwebsrv`.

```
vboxwebsrv -H 0.0.0.0 -A null -b
```

```
lab3 git:(main) ✗ vboxwebsrv -H 0.0.0.0 -A null -b
Oracle VirtualBox web service Version 7.2.20
Copyright (C) 2007-2026 Oracle and/or its affiliates
00:00:00.002200 main     VirtualBox web service 7.2.20 r175154 darwin.arm64 (Sep 22 2026 09:56:36) release log
00:00:00.002201 main     Log opened 2026-11-06T07:34:10.518937000Z
00:00:00.002202 main     Build Type: release
00:00:00.002207 main     OS Product: Darwin
00:00:00.002209 main     OS Release: 25.6.0
00:00:00.002211 main     OS Version: Darwin Kernel Version 25.6.0: Fri Jul 31 19:18:43 PDT 2026; root:xnu-12377.161.14~5/RELEASE_ARM64_T8112
00:00:00.002581 main     Firmware type: failed - VERR_NOT_SUPPORTED
00:00:00.002938 main     Host RAM: 8192MB (8.0GB) total, 1814MB available
00:00:00.002950 main     Executable: /Applications/VirtualBox.app/Contents/MacOS/vboxwebsrv
00:00:00.002951 main     Process ID: 43484
00:00:00.002951 main     Package type: DARWIN_ARM64_GENERIC
lab3 git:(main) ✗
```

Рис. 13. Запуск веб-сервиса VirtualBox

Запустим phpVirtualBox.

```
docker run --name vbox_http --restart=unless-stopped \
  -p 8089:80 \
  -e ID_HOSTPORT=host.docker.internal:18083 \
  -e ID_NAME=MyVBoxServer \
  -e ID_USER=admin \
  -e ID_PW=admin \
  -e CONF_noAuth=true \
  -d joweisberg/phpvirtualbox:latest
```

```
lab3 git:(main) ✗ docker run --name vbox_http --restart=unless-stopped \
  -p 8089:80 \
  -e ID_HOSTPORT=host.docker.internal:18083 \
  -e ID_NAME=MyVBoxServer \
  -e ID_USER=admin \
  -e ID_PW=admin \
  -e CONF_noAuth=true \
  -d joweisberg/phpvirtualbox:latest
Unable to find image 'joweisberg/phpvirtualbox:latest' locally
latest: Pulling from joweisberg/phpvirtualbox
f6e3073a6dc2: Pull complete
d8ad8cd72600: Pull complete
d156013eedd8: Pull complete
95e1a6d62ba5: Pull complete
d64d1ca1626b: Pull complete
ccb524aab87d: Pull complete
1bffd79bf42e: Pull complete
2673f43e0852: Download complete
Digest: sha256:19ff001afc8f517e989c66bc3f02266829a27a7ff9b043d81e2d7f9d83e1c3f4
Status: Downloaded newer image for joweisberg/phpvirtualbox:latest
e643197a243c59c1ff398e8b95aa6bb512f9fdac2de23372547703ff4eebc650
lab3 git:(main) ✗
```

Рис. 14. Создание контейнера с phpVirtualBox

Логин:пароль от phpVirtualBox – `admin:admin`.

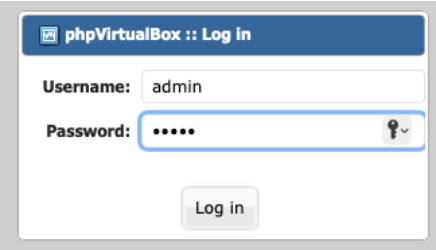


Рис. 15. Начальный экран phpVirtualBox

Можно увидеть те же виртуальные машины, что и в VirtualBox.

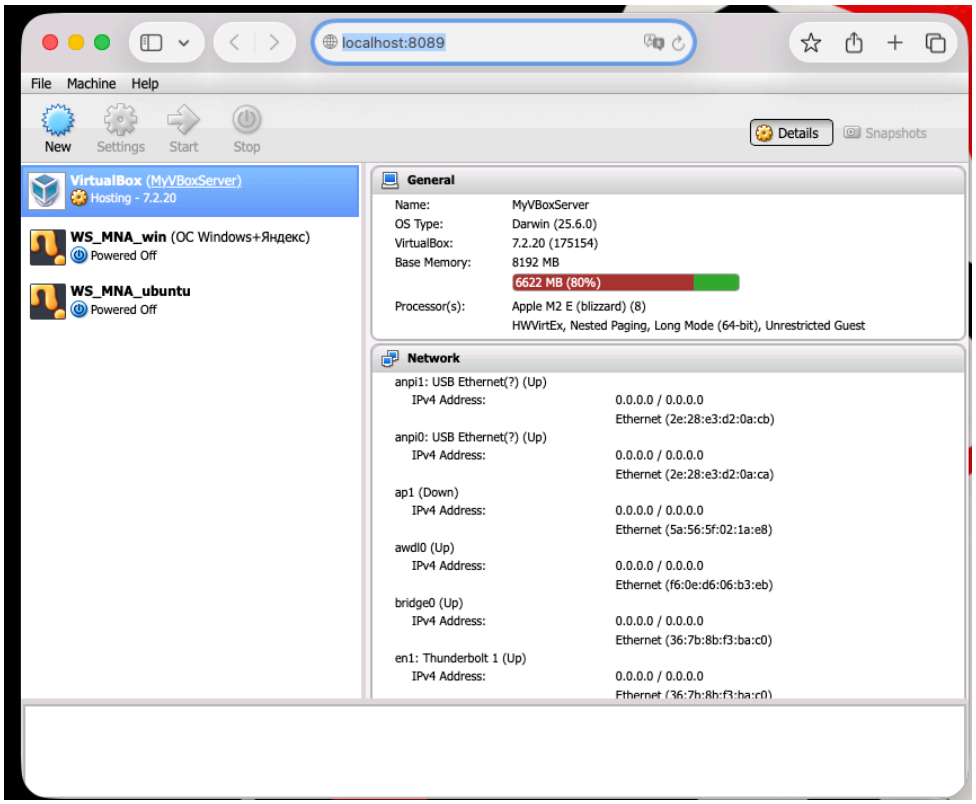


Рис. 16. Список ВМ в phpVirtualBox

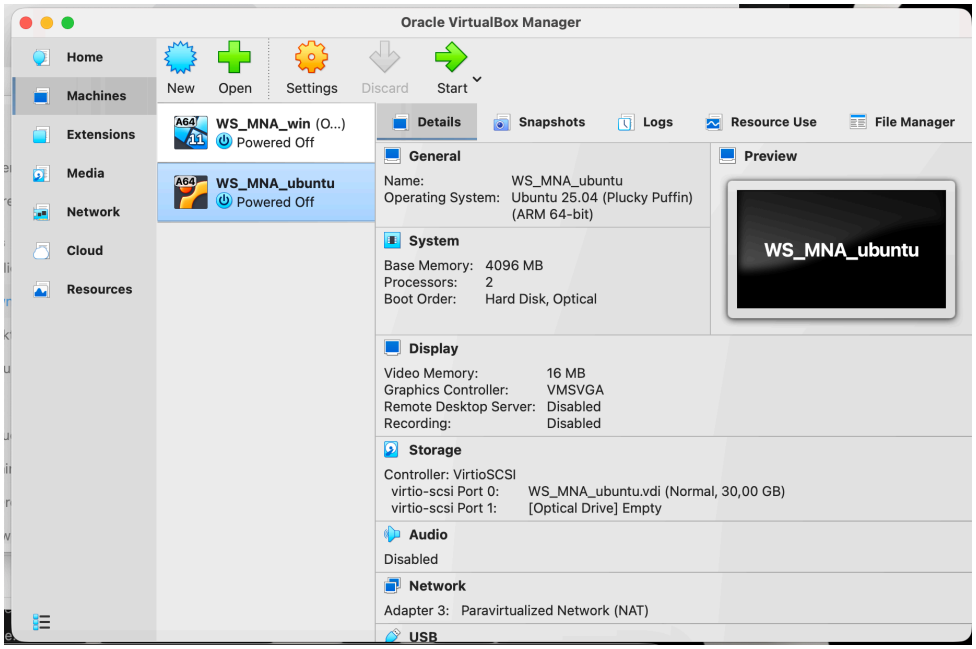


Рис. 17. Список ВМ в VirtualBox

8. Настройка окружения с помощью Docker Compose

Создан файл `docker-compose.yml`, описывающий три сервиса: `db` (MariaDB), `app` (NextCloud), `web` (Nginx).

```yaml
services:
  db:
    image: mariadb:12.1.1-rc
    container_name: nextcloud-mariadb
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: admin
      MYSQL_DATABASE: nextcloud
      MYSQL_USER: nextcloud_user
      MYSQL_PASSWORD: nextcloud_pass
    volumes:
      - /Users/admin/ws/docs/lab3/home/db:/var/lib/mysql
    networks:
      - nextcloud-network

  app:
    image: nextcloud:latest
    container_name: nextcloud-app
    restart: unless-stopped
    ports:
      - "2022:80"
    environment:
      MYSQL_HOST: db
      MYSQL_DATABASE: nextcloud
      MYSQL_USER: nextcloud_user
      MYSQL_PASSWORD: nextcloud_pass
      NEXTCLOUD_ADMIN_USER: admin
      NEXTCLOUD_ADMIN_PASSWORD: admin
      NEXTCLOUD_TRUSTED_DOMAINS: localhost
    volumes:
      - /Users/admin/ws/docs/lab3/home/nextcloud:/var/www/html
    depends_on:
      - db
    networks:
      - nextcloud-network

  web:
    image: nginx:alpine
    container_name: nginx-webserver
    restart: unless-stopped
    ports:
      - "8080:80"
    volumes:
      - /Users/admin/ws/docs/lab3/home/www/html:/usr/share/nginx/html:ro
    depends_on:
      - app
    networks:
      - nextcloud-network

networks:
  nextcloud-network:
    driver: bridge
```

Запустим сервисы и проверим их статус.

```
docker compose up -d
docker compose ps
```

```
● ➜  lab3 git:(main) ✗ docker compose up -d
  [+] up 4/4
   ✔ Network lab3_nextcloud-network Created
   ✔ Container nextcloud-mariadb    Started
   ✔ Container nextcloud-app        Started
   ✔ Container nginx-webserver      Started

  What's next:
      Filter, search, and stream logs from all your Compose services
      in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs
● ➜  lab3 git:(main) ✗ docker compose ps
  NAME                IMAGE                COMMAND                  SERVICE   CREATED          STATUS          PORTS
  nextcloud-app       nextcloud:latest     "/entrypoint.sh apac…"   app       7 seconds ago    Up 5 seconds    0.0.0.0:2022->80/tcp, [::]:2022->80/tcp
  nextcloud-mariadb   mariadb:12.1.1-rc    "docker-entrypoint.s…"   db        7 seconds ago    Up 6 seconds    3306/tcp
  nginx-webserver     nginx:alpine         "/docker-entrypoint.…"   web       7 seconds ago    Up 5 seconds    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
○ ➜  lab3 git:(main) ✗
```

Рис. 18. Запуск сервисов и проверка их статуса через Docker Compose

Страницу NextCloud можно найти по адресу http://localhost:2022.

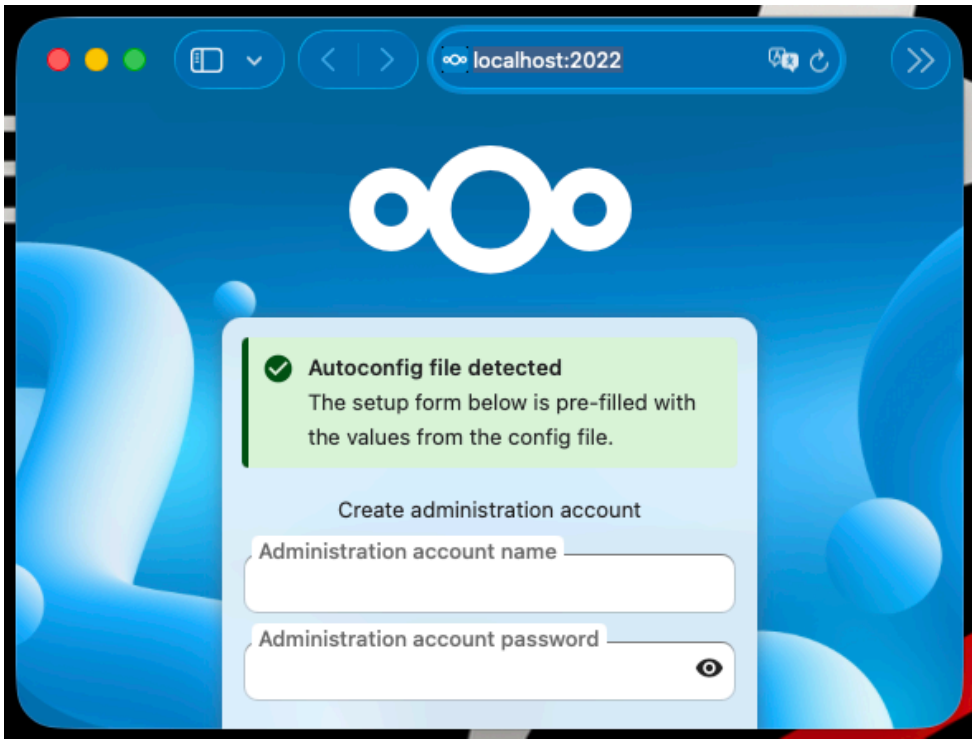


Рис. 19. Доступный через порт 2022 NextCloud

Судя по содержимому нового контейнера `mariadb` — инициализация прошла успешно.

```
→  lab3 git:(main) ✗ docker exec -it nextcloud-mariadb mariadb -u root -padmin
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 12.1.1-MariaDB-ubu2404 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> use mariadb;
ERROR 1049 (42000): Unknown database 'mariadb'
MariaDB [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| malyshev           |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.011 sec)
```

Рис. 20. Таблицы NextCloud в MariaDB

9. Регистрация на Docker Hub

Выполнена регистрация на https://hub.docker.com.

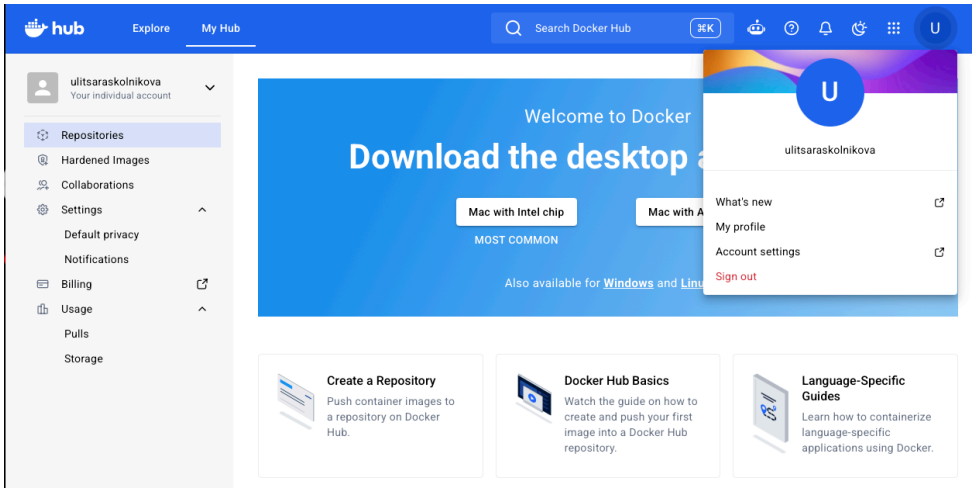


Рис. 21. Мой профиль на Docker Hub

10. Публикация образа на Docker Hub

Образ `my-nginx` помечен тегом и отправлен в публичный репозиторий.

```
● ➜  lab3 git:(main) ✗ docker tag my-nginx:latest ulitsaraskolnikova/my-nginx:latest
● ➜  lab3 git:(main) ✗ docker push ulitsaraskolnikova/my-nginx:latest
  The push refers to repository [docker.io/ulitsaraskolnikova/my-nginx]
  a6fb6399df14: Pushed
  1b5b466c140b: Pushed
  b90627b8f429: Pushed
  8b0f6b81d522: Pushed
  5faf28e117a4: Pushed
  89c6596257b9: Pushed
  5276acf80473: Pushed
  66eb34db8c04: Pushed
  f4a8c3eaf855: Pushed
  b25486411ab9: Pushed
  4f4fb700ef54: Mounted from library/nextcloud
  latest: digest: sha256:5c3e4f244248bda523cb04a6cc5d28725292675262bfcc782589a593aafc82ce size: 856
○ ➜  lab3 git:(main) ✗
```

Рис. 22. Push образа в Docker Hub

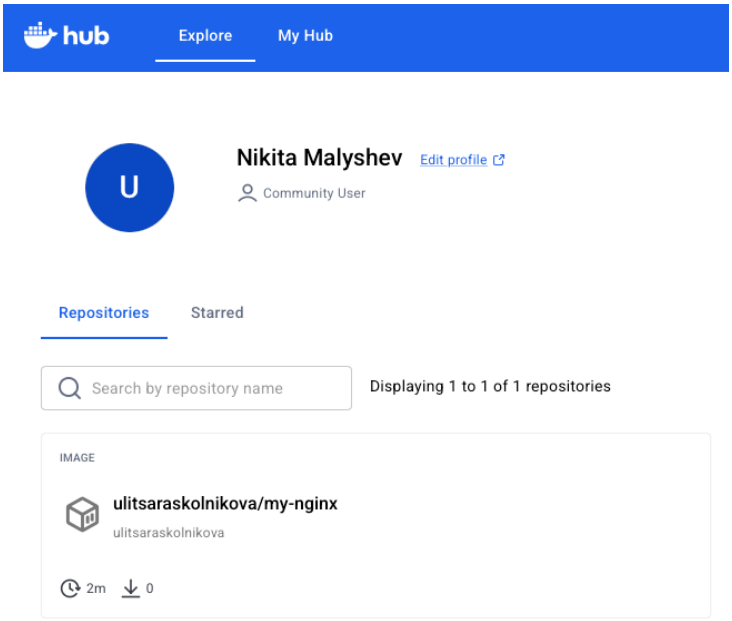


Рис. 23. Образ, опубликованный в публичном репозитории Docker Hub

11. Запуск приложения из Docker Hub

Запущен публичный образ игры 2048. Приложение доступно в браузере по порту 8082.

```
docker run -p 8082:80 cnrock/2048
```

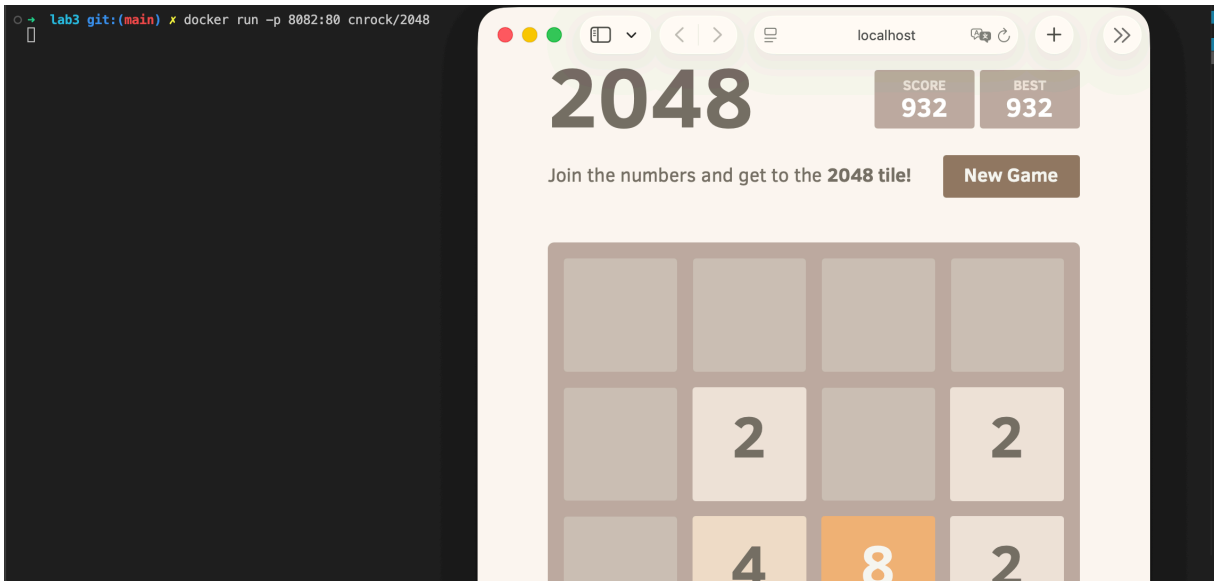


Рис. 24. Стороннее приложение из Docker Hub, запущенное в браузере

12. Основные команды Docker

В ходе выполнения лабораторной работы были применены следующие команды Docker для управления контейнерами, образами и многоконтейнерными приложениями.

Управление контейнерами:

- `docker create` — создаёт контейнер из образа без его запуска.
- `docker start` — запускает ранее созданный или остановленный контейнер.
- `docker run` — создаёт и сразу запускает новый контейнер из образа.
- `docker ps` — отображает список только запущенных контейнеров.
- `docker ps -a` — отображает все контейнеры, включая остановленные.
- `docker inspect` — выводит подробную информацию о контейнере (сетевые настройки, тома, переменные окружения и др.).
- `docker logs` — показывает журнал (логи) работы контейнера.
- `docker stop` — корректно останавливает контейнер, отправляя сигнал завершения (SIGTERM, затем при необходимости SIGKILL).
- `docker kill` — немедленно завершает работу контейнера принудительно (SIGKILL).
- `docker rm` — удаляет остановленный контейнер из системы.
- `docker exec` — выполняет указанную команду внутри уже запущенного контейнера.

Управление образами:

- `docker build` — собирает образ на основе инструкций из файла Dockerfile.

- `docker pull` — загружает образ из реестра (например, с Docker Hub).
- `docker push` — отправляет локальный образ в удалённый реестр.
- `docker images` (или `docker image ls`) — отображает список всех локальных образов.
- `docker image inspect` — выводит детальную информацию о метаданных и конфигурации образа.
- `docker rmi` (или `docker image rm`) — удаляет указанный образ из локального хранилища.
- `docker login` — выполняет аутентификацию в Docker Hub или другом реестре контейнеров.

Команды Docker Compose:

- `docker compose up` — создаёт и запускает все сервисы, описанные в файле docker-compose.yml.
- `docker compose down` — останавливает и удаляет контейнеры, сети и тома, созданные через docker-compose up.
- `docker compose ps` — отображает статус всех сервисов в проекте.
- `docker compose logs` — выводит журналы всех запущенных сервисов.
- `docker compose build` — пересобирает образы, указанные в конфигурации Compose.

Прочие команды:

- `docker system df` — отображает использование дискового пространства Docker (образы, контейнеры, тома).
- `docker volume ls` — перечисляет все тома, созданные Docker.
- `docker network ls` — перечисляет все сети, управляемые Docker.
- `docker version` — показывает версии Docker Engine и других компонентов.
- `docker info` — выводит подробную информацию о конфигурации и состоянии Docker-окружения.

13. Работа с Portainer

13.1. Запуск

Добавим в `docker-compose.yaml` конфигурацию сервиса Portainer.

```
  portainer:
    image: portainer/portainer-ce:lts
    container_name: portainer
    restart: always
    ports:
```

```
      - "9443:9443"
      - "8000:8000"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - portainer_data:/data
    networks:
      - nextcloud-network

volumes:
  portainer_data:
    name: portainer_data
```

Запустим.

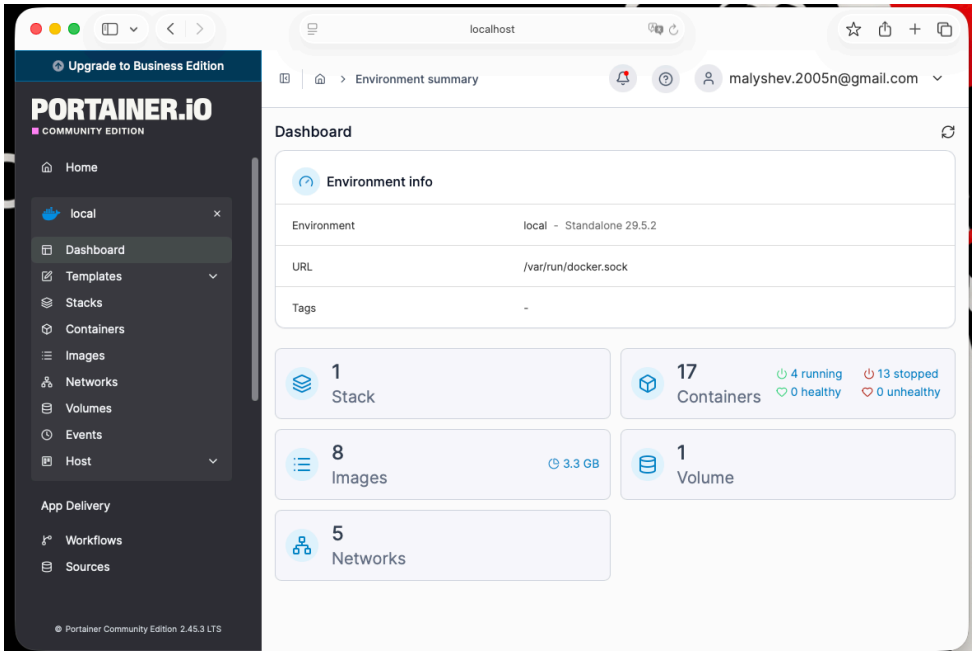


Рис. 25. Дашборд в Portainer

13.2. Возможности Portainer

13.2.1. Мониторинг

- **Обзор состояния** — дашборд с количеством контейнеров, образов, томов и сетей; статус каждого контейнера (running/stopped/error).
- **Логи в реальном времени** — просмотр stdout/stderr любого контейнера без `docker logs`.
- **Статистика ресурсов** — потребление CPU, RAM, сети и диска по контейнерам.
- **События Docker** — журнал событий движка (старт, стоп, ошибки, изменения).
- **Health-статусы** — отображение состояния healthcheck-проверок сервисов.

13.2.2. Администрирование

- **Управление контейнерами** — создание, запуск, остановка, перезапуск, удаление, просмотр деталей, выполнение команд внутри (`exec`).
- **Управление образами** — сборка из Dockerfile, pull, push, удаление, просмотр слоёв и метаданных.
- **Управление томами и сетями** — создание, удаление, просмотр подключений.
- **Стеки (Compose)** — развёртывание и редактирование `docker-compose.yml` прямо из UI, обновление стека.
- **Шаблоны приложений** — быстрое развёртывание популярных сервисов в пару кликов.
- **Права и роли** — пользователи, команды, разграничение доступа к окружениям (RBAC).
- **Мультиокружения** — управление несколькими Docker-хостами (локальными и удалёнными) из одного интерфейса.

13.2.3. Преимущества

- **Единая точка управления** — вместо десятков команд `docker` — визуальный интерфейс.
- **Снижение порога входа** — не нужно помнить синтаксис CLI для типовых операций.
- **Наглядность** — статусы, логи и метрики видны сразу, без ручного опроса.
- **Безопасность** — RBAC и централизованный доступ вместо прямого доступа к `docker.sock`.
- **Удалённое администрирование** — управление хостами через веб-браузер, в т.ч. Edge-агенты.
- **Воспроизводимость** — стеки Compose хранятся и версионируются в UI.
- **Бесплатность CE** — Community Edition покрывает большинство задач без лицензии.

14. Вывод

В ходе лабораторной работы я освоил Docker на практике: собрал собственный образ Nginx с HTML-страницей через `Dockerfile`, запустил MariaDB и создал базу через `docker exec`, поднял NextCloud с пользователями, связал phpVirtualBox с локальным `vboxwebsrv`. Описал три сервиса в `docker-compose.yml` и запустил всё одной командой, а затем опубликовал образ `my-nginx` на Docker Hub и запустил стороннее приложение оттуда же.

Главный вывод: Docker упрощает развёртывание — `compose up` заменяет ручную установку, тома сохраняют данные между перезапусками, Docker Hub позволяет делиться образами, а Portainer даёт наглядный GUI поверх CLI.